Kevin Küster

Zusammenfassung: Procedures & Functions

Procedures & Functions

Prozeduren & Funktionen

Ermöglicht die Zerlegung eines Programms in gekapselte Einzelteile

Prozeduren: Funktionen ohne Rückgabewert

Funktionen: Programmteile, die Bestimmten Wert zurückgeben
↳ Kann Teil einer Zuweisung sein

Feste Funktionsteile können lokal (unsichtlich für den Nutzer) wiederverwendet werden (Verschachtelung) ⇒ Lokalitätsprinzip

- Prozeduren werden durch einen Aufruf eingeleitet
- Prozeduren werden beendet

⇒ Statische Lösung (nur JMP-Befehle) nicht möglich, da eine abgearbeitete Prozedur ohne Weiteres nicht entscheiden kann, von wo sie aufgerufen wurde bzw. wo das Programm nach Prozedur-Abarbeitung fortfahren soll

↳ Aufrufende Prozedur legt entsprechende Adresse (unmittelbar nachfolgende) auf Stack und speichert damit Programm-Zustand

Beispiel

Code-Hochsprache:

```
procedure PrintPrompt() begin
    ClearScreen();
    WriteLn("=>");
end procedure

procedure PromptInput() begin
    PrintPrompt();
    ClearScreen();
    ProcessInput();
end procedure
begin
    PromptInput();
end
```

⇒

```
PrintPrompt:
    [...]   ; 0-A
    CALL ClearScreen
    [...]   ; 0-B
    CALL WriteLn
    [...]   ; 0-C
    RET
PromptInput:
    [...]   ; 1-A
    CALL PrintPrompt
    [...]   ; 1-B
    CALL ClearScreen
    [...]   ; 1-C
    CALL ProcessInput
    [...]   ; 1-D
    RET
start:
    [...]   ; 2-A
    CALL PromptInput
    [...]   ; 2-B
```

Befehle:

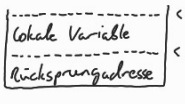
- CALL: gibt unmittelbar nachfolgende Adresse auf Stack und springt zu Sprungzieladresse
- RET: nimmt oberste Adresse vom Stack und setzt Befehlszeiger auf diese

Lokale Variablen

Variable dessen Lebzeit sich auf die Laufzeit eines Prozesses bezieht

⇒ Wird ebenfalls auf den Stack abgelegt,

↳ Verwendung zusätzlicher function Pointer (FP)

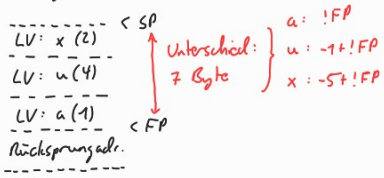


- Rücksprungadresse liegt 4 Byte vor FP-Register
  ↳ -4+!FP ("offset")

Beispiel

Lokale Variablen + Größe:

a: Byte, u: Wort, x: Short:



Sicherung von Rechenzuständen

Befehle:

- PUSHR: legt den Wert aller Register am Stack ab, angefangen mit akt. Adr. von SP
- POPR: liest SP und stellt Register anhand des Speicherbereichs wieder her.

```
PUSHR
PUSH p-n
⇒ PUSH p-i
CALL
POP
POP
POPR
```

Prozeduren mit Parameter

- Formalparameter: Wird durch Prozedur spezifiziert
- Aktualparameter: Werden bei Prozeduraufruf übergeben und an Formalparameter gebunden
  ↳ Bindung erfolgt anhand Stelligkeit (von caller an callee)

Code-Hochsprache:

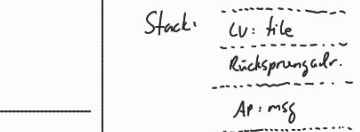
```
procedure Log(msg: String) is
    var file: File;          <- Formalparameter
begin
    file := open("~/data.log");
    if file <> NIL then
        WriteLn(file, msg); close(file);
    else
        WriteLn("error"); Exit(-1);
    end if;
end procedure;
```

```
begin
    Log("doSomething - before")
    [...]
    doSomething();           <- Aktualparameter
    [...]
    Log("doSomething - After")
```

⇒ Parameter: msg

Sicht Callee (Übernahme):

Stack:



⇒ Parameter in relativer Adressierung zu FP
(↳ msg: 8+!FP)

Sicht caller (Übergabe):

Parameter müssen ohne weiteren Eingriffen auf Stack gelegt werden vor Prozeduraufruf

↳ PUSH p1 → ("doSomething - before")
Call Log

Speicherabbild nach Rücksprung: Prozedur setzt SP zurück auf Wert von FP und führt Befehl RET aus

Mehrere Parameter: Reihenfolge umkehren → erster Parameter der Übergabe liegt oben auf Stack

```
⇒ PUSH p-n
  PUSH [...]
  PUSH p-0
  CALL [...]
```

Funktion

Prozedur mit Rückgabe => Realisiert durch zusätzlichen, künstlich eingeführten Parameter → Kennzeichnung durch Befehl: CLEAR

=> Beachtung an welcher Stelle Ergebnis einer Prozedur auf den Stack geschoben wird

Beispiel

Aufruf Fibonacci-Funktion:

```
[...]
CLEAR W -!SP      ; Funktionsergebnis Leeren
MOVE W n, -!SP    ; Parameter n auf Stack
CALL fib          ; Aufruf Funktion (fib)
ADD W I4, SP      ; SP um Wort-Parameter reduzieren (4 Byte)
MOVE W +!SP, R0   ; Ergebnis in R0
```

CALL-BY-VALUE/REFERENCE

CBV: Bei Wertübergabe von Parameter wird Variableninhalt kopiert
↳ Änderung des ursprungswertes nicht möglich

CBR: Übergeben werden die Referenzen der Quellen:
↳ Adressen auf CPU => Entsprechenden Speicherplatz reservieren
↳ Vom Caller müssen Referenzen (statt Werte) aufgerufen werden
↳ Vom Callee muss Referenz dereferenziert werden

Beispiel - Funktion schreiben

Code-Hochsprache:

```
function addiere(a:Integer, b:Integer) : Integer is
   Summe : Integer;
begin
   Summe: a+b;
   addiere := Summe;
end addiere
```

Stack-Frame:

Element	Typ	Größe	Offset zu R13
(LV) Summe	Integer	4 Bytes	-4
Gespeicherte Register	-	64 Bytes	0 bis 60
Rücksprungadr.	-	4 Bytes	unter Register
(p) a	Integer	"	64
(p) b	Integer	"	68
Rückgabewert	Integer	"	72

MI-Code-Funktion:

```
addiere:
   PUSHR                        ; Register sichern
   MOVE W SP, R13               ; Frame Pointer setzen
   SUB W W#SP, I4, SP           ; Platz LV: summe
   ADD W 64+!R13, 68+!R13, -4+!R13 ; summe = a+b
   MOVE W -4+!R13, 72+!R13      ; Rückgabewert setzen
   MOVE W R13, SP               ; SP auf FP
   POPR                         ; Register wiederherstellen
   RET                          ; Zurück zum Caller
```

Aufruf:

```
CLEAR W -!SP          ; Platz für Rückgabewert
MOVE W I7, -!SP   → b
MOVE W I5, -!SP   → a   } addiere(5,7)
CALL addiere
ADD W SP, I8, SP      ; Parameter entfernen
MOVE W !SP+, ergebnis
```

Beispiel Call-By-Reference - SWAP

Code-Hochsprache:

```
procedure swap(x: in out Integer, y: in out Integer) is
   tmp: Integer;
begin
   tmp := *x;
   x := y;
   y := tmp;
end swap;
```

MI-Code:

```
a:  DD W 42      ; Globale Var
b:  DD W 17

main:
   MOVEA a, -!SP
   MOVEA b, -!SP
   CALL swap
   ADD W SP, I8, SP
   HALT

swap:
   PUSHR
   MOVE W SP, R13
   SUB W SP, I4, SP      ; Platz für tmp

   MOVE W 64+!R13, R0    ; Adr. x nach R0
   MOVE W !R0, -4+!R13   ; Adr. R0 nach tmp

   MOVE W 68+!R13, R1    ; Adr. y → R1
   MOVE W !R1, !R0       ; *y nach *x

   MOVE W -4+!R13, !R1   ; tmp → y

   MOVE W R13, SP
   POPR
   RET
```

a = 42 → a = 17
b = 17 → b = 42